

BrokerVerse OOTB

Insurance broking platform by iorta TechNXT

Solution Architecture · Document 05 of 11

Shared Service Components

Product	BrokerVerse OOTB
Document	05 Shared Service Components
Version	1.1
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse OOTB Solution Architecture: 05 Shared Service Components
Document ID	BV-SA-05
Version	1.1
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, deploy/, docker-compose.yml, .github/workflows, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue
1.1	29 September 2026	iorta TechNXT	Updated to the current system: placement journey, co-insurance, posting rules, period end and BIR tax, bank reconciliation, document numbering, commission rate matrix, single PDF engine, uploads and go-live imports, housekeeping, business time zone, broker roles, deployment on the BrokerVerse URL

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

1 Introduction.....	4
1.1 Purpose and scope.....	4
2 Backend shared services.....	6
2.1 Overview.....	6
2.2 Configuration (src/config.js).....	7
2.3 Route registry (lib/registry.js).....	7
2.4 Authentication and authorisation (lib/auth.js and related).....	7
2.5 Record scope (lib/scope.js).....	8
2.6 Settings (lib/settings.js).....	8
2.7 Document numbering (lib/numbering.js).....	8
2.8 Posting engine (modules/accounting/lib).....	8
2.9 Maker-checker and audit.....	9
2.10 Notifications and audience.....	9
2.11 E-mail outbox (lib/mailer.js).....	9
2.12 PDF engine and letterhead (lib/pdf, lib/letterhead.js).....	9
2.13 Spreadsheets, CSV and upload templates.....	10
2.14 Reports engine (modules/reports).....	10
2.15 File storage and signed links.....	10
2.16 Scheduler and housekeeping (jobs/).....	10
2.17 Rate limiting and upload limits.....	11
2.18 Dates, money and validation.....	11
3 Front-end shared components.....	12
4 Related documents.....	13
5 Glossary.....	14

1 Introduction

1.1 Purpose and scope

This document describes the shared service components of BrokerVerse OOTB: the backend services in `backend/src/lib` and `backend/src/jobs` that every business module uses, the shared engines inside the larger modules (posting rules, reports), and the shared front-end components and utilities in `brokerverse/src`. For each component it gives the responsibility, the main interface, the configuration it reads and the points to observe when extending or operating it. New modules use these components and do not re-implement them; a backend test fails if a second copy of the date, amount, CSV, template or maker-checker helpers appears.

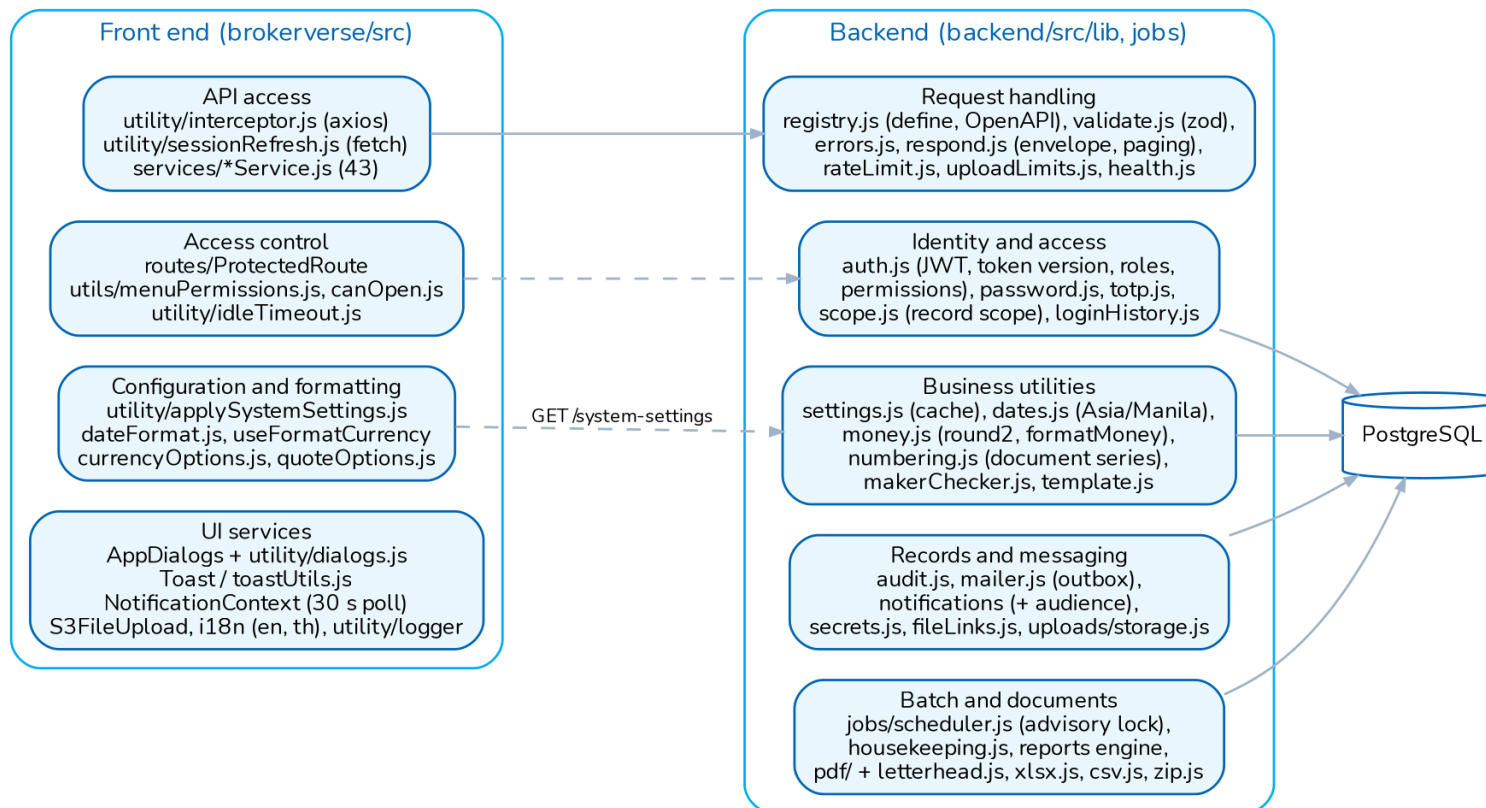


Figure 1. Shared service components of the front end and the backend

2 Backend shared services

2.1 Overview

Component	Location	Responsibility
Configuration	src/config.js	Environment configuration and the production start-up check
Route registry	lib/registry.js	Declares every endpoint, enforces sign-in, records roles, permissions and the screen for the API documentation
Authentication and authorisation	lib/auth.js, lib/password.js, lib/totp.js, lib/loginHistory.js	Tokens, session revocation, roles with inheritance, permissions, password policy, two-factor, sign-in log
Record scope	lib/scope.js	Own-book scoping for roles listed in <code>security.scoped_roles</code> (none by default)
Settings	lib/settings.js	Configuration from <code>app_settings</code> with a cache that follows changes on every instance
Document numbering	lib/numbering.js, SQL <code>next_document_number()</code>	Numbers of every document series from the Document Numbering master
Posting engine	modules/accounting/lib/posting.js, ledger.js, coinsurance.js	Journals from posting rules and account roles; co-insurance split; period checks
Maker-checker	lib/makerChecker.js	The approver must differ from the maker
Audit	lib/audit.js	Audit log of every change
Notifications	modules/notifications/service.js <code>notify()</code>	In-app notifications to a user or to a permission audience
E-mail outbox	lib/mailer.js, lib/template.js	Queue, render and deliver e-mail
PDF engine and letterhead	lib/pdf, lib/letterhead.js, modules/documents/templates.js	Every document and report PDF, with the Company master letterhead
Spreadsheets and files	lib/xlsx.js, lib/csv.js, lib/zip.js, modules/documents/uploadTemplates.js	XLSX and CSV output, upload templates
Reports engine	modules/reports/engine.js, queries.js, service.js	Parameterised report SQL, on-screen runs, files, schedules
File storage and signed links	modules/uploads/storage.js, fileTypes.js, lib/fileLinks.js, lib/secrets.js	S3-style key space on disk, type checking, signed expiring URLs
Scheduler and housekeeping	jobs/scheduler.js, jobs/handlers.js, jobs/housekeeping.js	Cron jobs in the business time zone with advisory locks and run history; retention purge
Rate limiting and upload limits	lib/rateLimit.js, lib/uploadLimits.js	Sign-in and global API limits; multipart and import limits
Dates and money	lib/dates.js, lib/money.js, lib/birthDate.js	Business dates in the configured time zone; rounding and formatting
Validation, errors and responses	lib/validate.js, lib/errors.js, lib/respond.js	zod validation, error envelope, success envelope and paging
Request context and logging	lib/requestContext.js, lib/logger.js	The signed-in user deep inside a request; pino logger with redaction
Health	lib/health.js, modules/system	Readiness, liveness, version

Component	Location	Responsibility
Database access	db/pool.js, db/migrate.js	Pool, query, one, many, withTransaction; migrations under an advisory lock

2.2 Configuration (src/config.js)

`buildConfig()` reads the environment once (with `.env` through `dotenv` in development): `PORT` (8000), `DATABASE_URL`, `JWT_SECRET`, `JWT_ACCESS_TTL_SECONDS` (1,800), `JWT_REFRESH_TTL_SECONDS` (2,592,000), `DATA_ENCRYPTION_KEY`, `CORS_ORIGINS`, `UPLOAD_DIR`, `PUBLIC_BASE_URL`, `SMTP_URL`, `LOG_LEVEL`, `FILE_URL_TTL_SECONDS` (1,800), `JSON_BODY_LIMIT` (2 MB), `UPLOAD_MAX_MB` (10), `UPLOAD_MAX_FILES` (10), `IMPORT_MAX_MB` (10), `IMPORT_MAX_INFLATED_MB` (50), `IMPORT_MAX_ROWS` (20,000). Operational variables are read where they are used: `SEED_SAMPLE_DATA`, `SCHEDULER_ENABLED`, `SCHEDULER_RELOAD_SECONDS` (30), `SETTINGS_CHECK_MS` (5,000), `SETTINGS_CACHE_TTL_MS` (60,000), `MIGRATION_LOCK_TIMEOUT_MS` (600,000), `ADMIN_PASSWORD`, `HEALTH_DB_TIMEOUT_MS` (2,000), `SHUTDOWN_TIMEOUT_MS` (25,000) and `GIT_COMMIT`. With `NODE_ENV=production`, `assertProductionConfig()` stops the process when a secret is missing, a placeholder, shorter than 32 characters or reused, when `CORS_ORIGINS` is `*`, or when `PUBLIC_BASE_URL` is missing or local. The full list with comments is `backend/.env.example`; the production template is `deploy/backend.env.example`.

2.3 Route registry (lib/registry.js)

Every module creates its router with `moduleRouter(module, prefix)` and declares each endpoint with `define({ method, path, summary, auth, roles, permissions, screen, request, query, response, middleware, handler }).define()`:

- places `requireAuth` first unless the route is declared `auth: false`, so no route can be left unauthenticated by mistake;
- wraps the async handler so errors reach the error middleware;
- records the route, the roles and permissions of its middleware, the front-end screen and example payloads in `ROUTES`. `npm run export:api` turns them into the OpenAPI document, the Postman collection and the Excel and CSV touchpoint list, whose "By Screen" sheet leads support from a screen to its API calls.

2.4 Authentication and authorisation (lib/auth.js and related)

Function	Behaviour
<code>signAccess(user, { restrict })</code>	HS256 access token with the effective roles, permissions and token version; restricted tokens for two-factor enrolment or a forced password change
<code>signRefresh(user, jti), signToken()</code>	Refresh token (recorded in <code>refresh_tokens</code>); other short-lived tokens (two-factor challenge, quotation approval, report download) with their own type claim
<code>verify()</code>	Accepts HS256 only
<code>authenticate(), requireAuth</code>	Token type, active user and unchanged token version; restricted tokens only on their allowed paths
<code>loadUser()</code>	Reads the user with the effective roles of <code>user_effective_roles()</code> : assigned roles plus inherited ones (Accounting Manager inherits Accounting)
<code>revokeSessions(userId, { refresh })</code>	Raises <code>token_version</code> ; optionally revokes all refresh tokens
<code>requirePermission(...codes), requireRole(...roles), isAdmin(), hasPermission()</code>	Authorisation middleware; the administrator role is named once (<code>ADMIN_ROLE = 'system-admin'</code>) and passes every check

Function	Behaviour
lib/password.js	Policy from <code>security.password_*</code> ; history of the last N hashes; expiry after <code>security.password_max_age_days</code>
lib/totp.js, lib/secrets.js	RFC 6238 TOTP (any authenticator app); AES-256-GCM encryption of the TOTP secret; HMAC-hashed reset codes
lib/loginHistory.js	Records every password, two-factor and refresh attempt with IP, user agent and reason

2.5 Record scope (lib/scope.js)

A user is scoped when every role they hold is listed in `security.scoped_roles`. The setting is empty in the seed (the agent sign-in was withdrawn), so today no user is scoped. The mechanism stays for a broker who wants account executives to see only their own book: list services add `scopeSql(scope, entity, alias, params)` to their WHERE clause; detail and action routes use the `ownRecord(entity)` middleware, which answers 404 for a record the user may not see.

2.6 Settings (lib/settings.js)

`getSetting(key, fallback)` reads `app_settings.value` (JSONB) and caches it in the process. `getSettings(group)` lists a group; `setSetting()` updates the row, stamps `updated_by` and `updated_at` and drops the key from the local cache. Every business parameter is read this way; the code fallback must equal the seeded value (tested).

The cache follows changes made through any instance. At most every `SETTINGS_CHECK_MS` (5 seconds) a read compares the table version (row count and latest `updated_at`) with the one it saw last and drops the whole cache when it moved. Each entry also expires after `SETTINGS_CACHE_TTL_MS` (60 seconds), which covers changes made outside the API. No restart is needed after a configuration change.

2.7 Document numbering (lib/numbering.js)

`nextDocumentNumber(code, { db, branch, lob, date, unique })` returns the next number of a series of the Document Numbering master, for example `POL-2026-00001`. Called with the caller's transaction (`db`), the counter row stays locked until the transaction ends and a rollback gives the number back. The date defaults to today in `general.timezone`. With `unique: { table, column }` a number already present in the target column (entered by hand or imported) is skipped. Series, patterns and reset rules are described in document 02; `src/modules/document-numbering/README.md` has the support notes.

2.8 Posting engine (modules/accounting/lib)

`postEvent(eventCode, context, { db, user })` turns a business event into a journal. It loads the posting rule version in force on the posting date, resolves every line to a GL account (a fixed account, the account role in `accounting.account.<role>`, a resolver, or an account the caller passes) and calls `createJournal`, which numbers the journal, checks that debits equal credits, that the accounts are active and that the period accepts the posting (`assertPostingAllowed` of the period-end module), and posts it. Lines marked per participant are split across co-insurers by share (`coinsurance.js`, remainder on the lead). Posting rules are edited on Master > Finance > Posting Rules, which has a Simulate button; the list of events is `EVENTS` in `posting.js`, and `test/posting-rules.test.js` checks that every event balances on its sample.

2.9 Maker-checker and audit

`assertChecker(user, makerId, what)` refuses an approval by the user who created or submitted the record. The switch `finance.maker_checker_enabled` turns it off for a small office with one accounting user; reinsurance treaties and cessions keep the rule always, and the period close and bank reconciliation check the approver themselves.

`audit(req, { entity, entityId, action, before, after })` inserts into `audit_log` the user id and name, entity, id, action, before and after JSON and the client IP. Mutating handlers call it after the change; it runs as its own statement, not inside the business transaction. Reviews: Master > Audit Trail (`GET /api/settings/audit`, permission `read:audit`, filters entity, entity id, username and a from / to business-date range; newest 200, at most 1,000).

2.10 Notifications and audience

`notify({ userId, type, priority, title, message, link, entity, entityId, audience })` inserts a row in `notifications`. A notification goes to one user or, without a user, to everyone who holds the permission in audience (for example `write:journal-vouchers` for journal vouchers awaiting approval). The front end polls the unread count every 30 seconds while signed in. Switches: `notification.approval_requests`, `notification.claim_status`, `notification.renewal_reminder`.

2.11 E-mail outbox (lib/mailer.js)

`queueEmail({ to, cc, subject, html, template, entity, entityId })` inserts into `email_outbox`; nothing is sent in the request. The `email-outbox` job calls `sendQueuedEmails()` every 5 minutes. When `notification.email_enabled` is true and `SMTP_URL` is set, it sends up to 50 queued messages (oldest first) from `notification.from_address`, marks them sent, or increments `attempts` and stores the error; after 5 failed attempts the message is `failed`. Templates are settings (`email.template.*`, `security.reset_email_*`, `reports.email_*`, `remittance.*_email_*`) rendered by `renderTemplate()` with `{{placeholder}}` substitution; the body is HTML-escaped and the subject is plain text.

Note: E-mail is off in the seed (`notification.email_enabled = false`). Until it is switched on, messages wait in the outbox with status `queued` and are sent afterwards. Throughput is 50 messages per run, 600 per hour; bulk renewal notices use their own queue job (`renewal-queue`, every minute). The sender must be the Office 365 mailbox of `SMTP_URL`, otherwise Office 365 refuses the message.

2.12 PDF engine and letterhead (lib/pdf, lib/letterhead.js)

One PDF engine, with no third-party dependency, produces every document and report: quotations, policy schedules, broker slips, placement slips, billing statements, official receipts, vouchers, debit notes, claim letters, reconciliation statements and report listings. `renderPdf(spec)` fills in the letterhead, formats and user and builds the file; `renderPdfBatch()` puts several documents in one file (bulk receipt and voucher prints); `renderReportPdf()` lays out a report with its parameters, columns and summary figures. Every page carries the letterhead of the primary active company of the Company master (`master_records` of type `company` flagged as the letterhead company) and a footer with the company, "Generated <date time> by <user>" and "Page X of Y". The logo comes from the company record, else from `documents.default_logo_path`. Dates follow `general.date_format`, amounts `currency.decimals`, times `general.timezone`. Text uses the PDF base fonts (Helvetica, WinAnsi); the peso sign prints as "PHP". The letterhead is cached for 30 seconds and cleared when the Company master changes. Document layouts are in `modules/documents/templates.js`.

2.13 Spreadsheets, CSV and upload templates

Writer	Use
<code>lib/xlsx.js</code> with <code>lib/zip.js</code>	Multi-sheet XLSX with styled frozen header, auto-filter and typed cells (money, date, integer)
<code>lib/csv.js</code>	RFC 4180 CSV with UTF-8 BOM; every cell starting with <code>=</code> , <code>+</code> , <code>-</code> or <code>@</code> is guarded against spreadsheet formulas
<code>modules/documents/uploadTemplates.js</code> , <code>scripts/build-upload-templates.js</code>	The upload templates (Data, Columns and Instructions sheets) built from each importer's own column list; the same workbooks are downloadable on the screens with an Upload button and published in <code>docs/templates</code>

2.14 Reports engine (modules/reports)

The catalogue (`report_definitions`, 39 reports, including financial statements, BIR reports, placement pipeline, market response, co-insurance and bank reconciliation reports) defines the parameters, the base query name, default columns, roles and permission. `engine.js` normalises the screen parameters (dates in the business time zone, account executive, insurer, branch and client filters), wraps the base query from `queries.js` with parameterised filters, sorting, paging and totals, and validates column identifiers. `service.js` runs reports on screen (`POST /reports/{code}/run`), generates files (CSV, XLSX, PDF; at most `reports.max_rows = 50,000` rows) into `UPLOAD_DIR/reports` recorded in `generated_reports`, returns download links signed for `reports.download_link_ttl_hours` (72 h), and runs report schedules as scheduled jobs that e-mail a download link. Generated files older than `reports.retention_days` (90) are deleted by the daily-reports job.

2.15 File storage and signed links

Function	Behaviour
<code>storeFile(file, { folder, userId, entity, entityId })</code>	Checks the file content against <code>uploads.allowed_types</code> (signature detection; the detected type is stored), reserves a key <code>folder/<timestamp>-<128-bit random>-<name></code> , writes the file under <code>UPLOAD_DIR</code> and records it in <code>documents</code>
<code>newKey()</code> , <code>reserveKey()</code> , <code>writeObject()</code>	One key generator for every upload and generated file; the two-step upload used by the presigned-URL endpoints
<code>resolveKey()</code>	Normalises the key and refuses paths outside <code>UPLOAD_DIR</code>
<code>publicUrl(key)</code> , <code>signedUrl(key)</code>	Canonical URL stored in records; signed URL with <code>exp</code> and HMAC <code>sig</code>
<code>lib/fileLinks.js</code>	Middleware that signs every object URL in JSON responses, so records keep unsigned URLs and links copied out of the application expire
<code>GET /s3/object/*</code>	Serves a file to a bearer-token caller or a valid signature; <code>nosniff</code> , sandbox CSP (PDF excepted), attachment disposition for anything other than images and PDF
Delete, overwrite	Only the uploader, a writer of the owning module or an administrator

The storage functions keep an S3-like key space, so they can later be replaced by Amazon S3 without changing the modules.

2.16 Scheduler and housekeeping (jobs/)

Aspect	Behaviour
Start	<code>startScheduler()</code> at API start (unless <code>SCHEDULER_ENABLED=false</code>) schedules every enabled row of <code>scheduled_jobs</code> with node-cron in the time zone <code>general.timezone</code> ; invalid cron expressions are skipped with a warning

Aspect	Behaviour
Changes	Every <code>SCHEDULER_RELOAD_SECONDS</code> (30) each instance compares a signature of the jobs table and the time zone with the one it loaded and reloads when it differs, so a schedule edited through one instance reaches all of them
Single execution	<code>runJob()</code> takes <code>pg_try_advisory_lock(hashtext('brokerverse.scheduled_job'), hashtext(code))</code> on a dedicated connection; if another instance holds it the run is skipped. A scheduled run is also skipped when a scheduled run of the same job already started in the same minute (clock skew between instances)
Recording	Each executed run inserts <code>job_runs</code> (triggered by <code>schedule</code> or the user name) and updates <code>scheduled_jobs.last_run_at</code> and <code>last_status</code> ; the handler output (JSON) or error is stored; a failure is also logged at error level with the job code and run id
Manual run	Master > Schedules > Run now (POST <code>/api/schedules/{code}/run</code>); listing needs <code>read:schedules</code>
Handlers	<code>renewalNotices</code> , <code>policyExpiry</code> , <code>quoteExpiry</code> , <code>receivableAgeing</code> , <code>dailyReports</code> , <code>emailOutbox</code> , <code>processRenewalQueue</code> , <code>renewalPipeline</code> , <code>collectionReminders</code> , <code>housekeeping</code> , <code>monthEndReminder</code> , <code>recurringJournals</code> , <code>accrualReversal</code> , <code>periodAutoSoftClose</code> , <code>bankAutoMatch</code> , <code>scheduledReport</code>
Housekeeping	<code>jobs/housekeeping.js</code> deletes, in batches of 5,000 rows, job runs, sent and failed e-mails, sign-in history, expired or revoked refresh tokens, used or expired reset codes, read notifications and completed queue items older than the days in System Settings > Housekeeping (<code>housekeeping.*; 0</code> keeps forever). The audit trail is kept unless <code>housekeeping.audit_log_days</code> is set, and never below seven years (document 10).

2.17 Rate limiting and upload limits

`lib/rateLimit.js` keeps two in-memory limiters per process: a sliding window for sign-in and forgot-password (`security.login_rate_limit`, 10 per 300 s, keyed by IP and by username) and a fixed-window global API limit (`security.api_rate_limit`, 600 requests per 60 s per signed-in user or per IP; health checks and CORS pre-flights are not counted; 429 with `Retry-After`). `lib/uploadLimits.js` provides `memoryUpload()` (documents and photos, per-file size and file count), `importUpload()` (one spreadsheet or statement file), `inflateEntry()` (caps decompression of workbook parts) and `assertRowLimit()`.

2.18 Dates, money and validation

Component	Interface
<code>lib/dates.js</code>	<code>businessTimeZone()</code> (<code>general.timezone</code>), <code>today()</code> , <code>businessDate()</code> , <code>nowInTz()</code> , <code>calendarPeriod(period)</code> , <code>isoDate()</code> , <code>addDays()</code>
<code>lib/money.js</code>	<code>round2()</code> (half away from zero with floating-point correction), <code>toNumber()</code> , <code>formatMoney(amount, currency)</code> using <code>currency.default</code> , its locale and <code>currency.decimals</code>
<code>lib/birthDate.js</code>	Age plausibility of dates of birth (<code>leads.min_age_years</code> , <code>leads.max_age_years</code>)
<code>lib/validate.js</code>	<code>validate(schema, part)</code> middleware with zod; the parsed value replaces the input
<code>lib/errors.js</code>	<code>HttpError</code> and helpers (<code>badRequest</code> , <code>unauthorized</code> , <code>forbidden</code> , <code>notFound</code> , <code>conflict</code>); <code>errorHandler</code> returns <code>{ success: false, message, requestId }</code> , with a generic message for 5xx
<code>lib/respond.js</code>	<code>ok()</code> , <code>created()</code> envelopes (list responses of the finance modules use <code>sendList()</code> in <code>modules/accounting/lib/http.js</code>); <code>paging()</code> (page and perPage with aliases, capped at 500, default 10); <code>pageMeta()</code>
<code>db/pool.js</code>	<code>query()</code> , <code>one()</code> , <code>many()</code> , <code>withTransaction(fn)</code> (BEGIN, COMMIT or ROLLBACK on one client)

3 Front-end shared components

Component	Responsibility
API access: utility/interceptor.js, utility/commonServices.js, utility/sessionRefresh.js, services/*Service.js	Axios client with the bearer token and a wrapper around <code>window.fetch</code> for the older services. On a 401 from a non-auth endpoint, one shared refresh per tab exchanges the refresh token (or picks up a token already rotated by another tab), retries the request, and ends the session with a redirect to <code>/login?session=ended</code> if the refresh fails. <code>REACT_APP_BASE_URL</code> (compiled in) is the API base.
Session: utility/tokenManager.js, logout.js, idleTimeout.js, redux/store.js	Token storage in <code>localStorage</code> ; sign-out clears the session keys and resets the Redux store; idle sign-out after <code>limits.session_idle_minutes</code> with a one-minute warning.
Route guard and menus: routes/ProtectedRoute, utils/menuPermissions.js, utils/canOpen.js	Deny-by-default role-to-menu map for the seven broker roles; <code>filterMenuForRoles()</code> and <code>isPathAllowed()</code> guard every route; <code>canOpen(path)</code> hides buttons that lead to screens the role cannot open, and approval actions the user may not take are hidden. The server remains the authority.
Dialogs and toasts: components/AppDialogs, utility/dialogs.js, components/Toast, utility/toastUtils.js	One shared toast and confirmation dialog, and an in-app prompt for reasons (reject, reverse, resolve), instead of the browser's alert, confirm and prompt.
System settings and branding: module/SystemSettings/store/systemSettingsSlice.js, utility/applySystemSettings.js	Loads the public system settings (<code>GET /api/system-settings</code> , also used by the sign-in page) and applies theme colours, favicon, logo, language, display currency, date format, phone format and quotation options. The default name is BrokerVerse with the iorta TechNXT logo.
Formatting: utility/dateFormat.js, utility/numberFormat.js, hooks/useFormatCurrency.js, utility/phoneFormat.js	Dates in <code>general.date_format</code> ; amounts in the configured currency and locale (<code>en-PH</code> grouping); Philippine mobile number format from settings.
Configuration-driven options: utility/currencyOptions.js, utility/quoteOptions.js, utility/systemCurrencies.js, master-option hooks	Drop-downs filled from settings and master data instead of constants.
Imports: services/importService.js, the Upload buttons of master screens, Chart of Accounts, Period Management and Collections	Upload of a template-based file, preview of the rows and the errors per row, download of the template.
Notifications: context/NotificationContext.js, components/NotificationDropdown, NotificationBadge	Unread count polled every 30 seconds while signed in; list, mark read, delete.
File upload: components/S3FileUpload	Upload with progress and preview through the <code>/s3</code> endpoints; shows the signed URLs returned by the API.
Internationalisation: <code>i18n.js</code> , locales/en.json, th.json	<code>i18next</code> with browser language detection; English complete (checked by <code>npm run check:i18n</code>); Thai partial; Filipino listed without a translation file.
Logging: <code>utility/logger.js</code>	Diagnostics silent in production builds, enabled per browser for support; ESLint warns on direct <code>console</code> calls.
Checks: <code>scripts/check-api-calls.js</code> , <code>scripts/check-translations.js</code>	<code>npm run check:api</code> compares every front-end API call with the backend OpenAPI document; <code>npm run check:i18n</code> compares the translation keys used with <code>en.json</code> .

4 Related documents

This document is one of eleven that together form the BrokerVerse OOTB Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components (this document)	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

5 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Advisory lock	PostgreSQL application-level lock (pg_try_advisory_lock, pg_advisory_lock) used so a job runs on one API instance at a time and only one instance applies migrations.
API	Application programming interface; here the BrokerVerse REST API under /api served by the Node.js backend.
Audit log / audit trail	Table audit_log: who changed which record, when, from which IP, with the before and after values.
BIR	Bureau of Internal Revenue (Philippines): tax authority whose rules drive record keeping and withholding tax certificates.
Co-insurance	Placement of one risk with several insurers, one of them the lead, each taking a share; premium, commission and claims are split by share.
Company master	Master record of the broker company; the primary company is the letterhead printed on every document and report.
CORS	Cross-Origin Resource Sharing: browser rule set that the API restricts to the configured web origins (CORS_ORIGINS).
Cron	Time-based schedule expression (minute hour day month weekday) used by the scheduled jobs.
CSP	Content Security Policy: HTTP header restricting what a page may load or run.
HMAC	Keyed-hash message authentication code; signs file links and hashes one-time codes.
Housekeeping	Daily job that deletes operational rows (job runs, sent e-mails, sign-in history, expired tokens, read notifications) after the retention days in System Settings.
HS256	HMAC-SHA256 JSON Web Token signature algorithm, the only one the API accepts.
JSONB	PostgreSQL binary JSON column type, used for flexible document parts of records.
JWT	JSON Web Token: signed token carrying the user id, roles, permissions and token version.
Maker-checker	Control in which a record created or submitted by one user must be approved by a different user.
Migration	Numbered SQL file in backend/src/db/migrations applied once, in order, on API start.
Office 365	Microsoft 365 mail service; BrokerVerse sends e-mail through smtp.office365.com with the mailbox in SMTP_URL.
Posting rule	Versioned definition of the journal lines for one business event (policy issued, receipt applied ...); every system journal is built from it.
Record scope	Rule that users holding only roles listed in security.scoped_roles see only their own records (lib/scope.js); no role is scoped by default.
Refresh token	Long-lived token (30 days) exchanged for a new access token; rotated on every use.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
Seed	Idempotent data load run on every API start: reference data always, sample data only when SEED_SAMPLE_DATA is on.
SMTP	Simple Mail Transfer Protocol; the API sends e-mail through the server in SMTP_URL.
Token version	users.token_version, carried in every access token; raising it ends all sessions of the user.
TOTP / 2FA	Time-based one-time password (RFC 6238) used for two-factor authentication.

Term	Meaning
zod	TypeScript-first schema validation library used to validate request bodies and parameters.